

# 自然语言处理（NLP）教程

## 第二节课：特征提取与文本分类

ClimbAI

2025 年 9 月 6 日

# 目录

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1 课程概述</b>                                            | <b>4</b>  |
| 1.1 学习目标                                                 | 4         |
| 1.2 课程安排                                                 | 4         |
| <b>2 特征提取理论与方法 (60 分钟)</b>                               | <b>4</b>  |
| 2.1 2.1 特征提取概述 (15 分钟)                                   | 4         |
| 2.1.1 什么是特征提取                                            | 4         |
| 2.1.2 特征提取方法分类                                           | 5         |
| 2.2 2.2 经典特征提取方法 (30 分钟)                                 | 5         |
| 2.2.1 词袋模型 (Bag of Words)                                | 5         |
| 2.2.2 TF-IDF (Term Frequency-Inverse Document Frequency) | 6         |
| 2.2.3 N-gram 模型                                          | 7         |
| 2.3 2.3 词向量方法 (15 分钟)                                    | 7         |
| 2.3.1 Word2Vec 简介                                        | 7         |
| 2.3.2 预训练词向量的使用                                          | 8         |
| <b>3 文本分类基础 (45 分钟)</b>                                  | <b>8</b>  |
| 3.1 3.1 文本分类任务概述 (15 分钟)                                 | 8         |
| 3.1.1 文本分类的定义                                            | 8         |
| 3.1.2 文本分类应用场景                                           | 8         |
| 3.2 3.2 文本分类流程 (15 分钟)                                   | 9         |
| 3.2.1 标准流程                                               | 9         |
| 3.2.2 各步骤详解                                              | 9         |
| 3.3 3.3 评估指标 (15 分钟)                                     | 10        |
| 3.3.1 混淆矩阵                                               | 10        |
| 3.3.2 核心评估指标                                             | 10        |
| 3.3.3 多分类评估                                              | 10        |
| <b>4 实战项目：中文情感分析系统 (75 分钟)</b>                           | <b>11</b> |
| 4.1 4.1 项目概述 (10 分钟)                                     | 11        |
| 4.1.1 项目目标                                               | 11        |
| 4.1.2 技术栈                                                | 11        |
| 4.2 4.2 数据准备与预处理 (20 分钟)                                 | 11        |
| 4.2.1 数据集准备                                              | 11        |
| 4.3 4.3 特征工程优化 (20 分钟)                                   | 17        |
| 4.3.1 高级特征提取技术                                           | 17        |
| 4.4 4.4 模型对比与选择 (15 分钟)                                  | 20        |
| 4.4.1 多模型对比实验                                            | 20        |
| 4.5 4.5 模型解释与分析 (10 分钟)                                  | 22        |

|          |                |           |
|----------|----------------|-----------|
| 4.5.1    | 模型可解释性分析       | 22        |
| <b>5</b> | <b>总结与扩展</b>   | <b>26</b> |
| 5.1      | 5.1 课程总结       | 26        |
| 5.2      | 5.2 进阶方向       | 26        |
| 5.3      | 5.3 实践建议       | 27        |
| <b>6</b> | <b>课后练习</b>    | <b>27</b> |
| 6.1      | 6.1 基础练习       | 27        |
| 6.2      | 6.2 进阶练习       | 28        |
| 6.3      | 6.3 项目练习       | 28        |
| <b>7</b> | <b>参考资料</b>    | <b>29</b> |
| 7.1      | 7.1 推荐阅读       | 29        |
| 7.2      | 7.2 在线资源       | 29        |
| 7.3      | 7.3 实验环境       | 30        |
| <b>8</b> | <b>附录</b>      | <b>30</b> |
| 8.1      | A.1 常用中文停用词表   | 30        |
| 8.2      | A.2 评估指标计算公式汇总 | 30        |
| 8.3      | A.3 常见问题与解决方案  | 31        |

# 1 课程概述

## 1.1 学习目标

### 知识目标 (Knowledge Objectives)

- 理解文本特征提取的基本概念和重要性
- 掌握经典特征提取方法：BoW、TF-IDF、N-gram
- 了解词向量的基本原理和应用
- 理解文本分类任务的框架和评估指标

### 技能目标 (Skill Objectives)

- 能够使用 Python 实现各种特征提取方法
- 掌握 scikit-learn 进行文本分类
- 能够评估和优化分类模型性能
- 学会处理中文文本分类任务

### 能力目标 (Competency Objectives)

- 能够根据任务选择合适的特征提取方法
- 具备构建端到端文本分类系统的能力
- 能够分析和解决文本分类中的常见问题
- 培养 NLP 项目的工程实践能力

## 1.2 课程安排

表 1: 第二节课时间安排

| 时间段    | 内容        | 时长     |
|--------|-----------|--------|
| 第 1 部分 | 特征提取理论与方法 | 60 分钟  |
| 第 2 部分 | 文本分类基础    | 45 分钟  |
| 第 3 部分 | 实战项目：情感分析 | 75 分钟  |
| 总计     |           | 180 分钟 |

## 2 特征提取理论与方法 (60 分钟)

### 2.1 2.1 特征提取概述 (15 分钟)

#### 2.1.1 什么是特征提取

文本特征提取是将非结构化的文本数据转换为机器学习算法可以处理的数值型特征向量的过程。

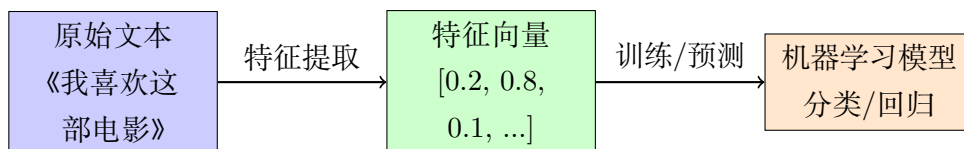


图 1: 文本特征提取流程

### 特征提取的重要性:

1. **机器理解文本**: 将人类语言转换为数学表示
2. **保留语义信息**: 尽可能保持文本的语义和语法信息
3. **降维与泛化**: 提取关键特征, 提高模型泛化能力
4. **计算效率**: 优化计算复杂度和存储空间

#### 2.1.2 特征提取方法分类

表 2: 文本特征提取方法分类

| 类别   | 方法          | 优点             | 缺点             |
|------|-------------|----------------|----------------|
| 统计方法 | BoW         | 简单直观           | 丢失词序信息         |
|      | TF-IDF      | 考虑词重要性         | 稀疏向量           |
|      | N-gram      | 保留局部顺序         | 维度爆炸           |
| 词向量  | Word2Vec    | 密集表示           | 无法处理 OOV       |
|      | FastText    | 子词信息           | 训练复杂           |
| 深度学习 | BERT<br>GPT | 上下文感知<br>生成能力强 | 计算开销大<br>需大量数据 |

## 2.2 经典特征提取方法 (30 分钟)

### 2.2.1 词袋模型 (Bag of Words)

词袋模型将文档表示为其包含词汇的集合, 忽略词汇的顺序和语法结构。

**数学定义**: 给定词汇表  $V = \{w_1, w_2, \dots, w_{|V|}\}$  和文档  $d$ , 词袋表示为:

$$BoW(d) = [c_1, c_2, \dots, c_{|V|}] \quad (1)$$

其中  $c_i$  是词汇  $w_i$  在文档  $d$  中的出现次数。

**示例**:

- 文档 1: 《我喜欢机器学习》
- 文档 2: 《机器学习很有趣》
- 词汇表: [我, 喜欢, 机器, 学习, 很, 有趣]

表 3: 词袋模型示例

| 文档   | 我 | 喜欢 | 机器 | 学习 | 很 | 有趣 |
|------|---|----|----|----|---|----|
| 文档 1 | 1 | 1  | 1  | 1  | 0 | 0  |
| 文档 2 | 0 | 0  | 1  | 1  | 1 | 1  |

优缺点分析:

- **优点:** 简单易懂, 计算高效, 适合短文本
- **缺点:** 丢失词序信息, 稀疏表示, 同义词处理困难

### 2.2.2 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF 是对词袋模型的改进, 考虑了词汇在文档中的重要性。

数学公式:

$$TF(t, d) = \frac{count(t, d)}{\sum_k count(k, d)} \quad (2)$$

$$IDF(t) = \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (3)$$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t) \quad (4)$$

其中:

- $TF(t, d)$ : 词汇  $t$  在文档  $d$  中的词频
- $IDF(t)$ : 词汇  $t$  的逆文档频率
- $|D|$ : 文档集合的大小
- $|\{d \in D : t \in d\}|$ : 包含词汇  $t$  的文档数量

**TF-IDF 的直观理解:**

- **TF 项:** 词汇在文档中出现越频繁, 越重要
- **IDF 项:** 词汇在整个语料库中越罕见, 越有区分性
- **平衡作用:** 既考虑局部重要性, 又考虑全局稀有性



图 2: TF-IDF 计算原理

### 2.2.3 N-gram 模型

N-gram 模型通过考虑连续的  $n$  个词来捕获局部的词序信息。

定义：

- **Unigram (1-gram)**: 单个词汇
- **Bigram (2-gram)**: 连续两个词汇
- **Trigram (3-gram)**: 连续三个词汇
- **N-gram**: 连续  $n$  个词汇

示例：句子：《自然语言处理很有趣》

表 4: N-gram 提取示例

| N-gram 类型 | 提取结果                 |
|-----------|----------------------|
| Unigram   | 自然, 语言, 处理, 很, 有趣    |
| Bigram    | 自然语言, 语言处理, 处理很, 很有趣 |
| Trigram   | 自然语言处理, 语言处理很, 处理很有趣 |

N-gram 的权衡：

- **$n$  值较小**: 特征稀疏度低, 但语义信息有限
- **$n$  值较大**: 语义信息丰富, 但特征维度急剧增长
- **实践经验**: bigram 和 trigram 的组合效果较好

## 2.3 词向量方法 (15 分钟)

### 2.3.1 Word2Vec 简介

Word2Vec 通过神经网络学习词汇的密集向量表示, 能够捕获词汇之间的语义关系。

**核心思想**: "You shall know a word by the company it keeps" - 通过上下文来理解词汇的含义。

两种架构：

1. **CBOW (Continuous Bag of Words)**: 通过上下文预测中心词
2. **Skip-gram**: 通过中心词预测上下文



图 3: Word2Vec 两种架构

词向量的特性：

- 语义相似性：语义相近的词向量距离较近
- 类比关系：  $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$
- 线性关系：词汇关系可以通过向量运算表达

### 2.3.2 预训练词向量的使用

常用的中文预训练词向量：

- 腾讯 AI Lab 词向量：800 万词汇，200 维
- 清华 THULAC 词向量：基于大规模中文语料
- fastText 中文向量：支持子词信息
- BERT 词向量：上下文感知的动态词向量

## 3 文本分类基础 (45 分钟)

### 3.1 3.1 文本分类任务概述 (15 分钟)

#### 3.1.1 文本分类的定义

文本分类是将文本文档自动分配到预定义类别中的监督学习任务。

数学形式化：给定训练集  $D = \{(d_1, y_1), (d_2, y_2), \dots, (d_n, y_n)\}$ ，其中：

- $d_i$ ：第  $i$  个文档
- $y_i \in \{c_1, c_2, \dots, c_k\}$ ：文档对应的类别标签

目标是学习一个分类函数  $f: D \rightarrow C$ ，使得对新文档  $d_{new}$  能正确预测其类别。

#### 3.1.2 文本分类应用场景

表 5: 文本分类应用领域

| 应用领域 | 具体任务   | 类别示例        |
|------|--------|-------------|
| 内容分析 | 新闻分类   | 政治、体育、科技、娱乐 |
|      | 主题分类   | 教育、医疗、金融、旅游 |
|      | 文档分类   | 合同、报告、邮件、简历 |
| 情感分析 | 产品评价   | 正面、负面、中性    |
|      | 社交媒体   | 喜悦、愤怒、悲伤、恐惧 |
| 风险控制 | 垃圾邮件检测 | 垃圾邮件、正常邮件   |
|      | 内容审核   | 违规、正常       |



## 3.2 3.2 文本分类流程 (15 分钟)

### 3.2.1 标准流程

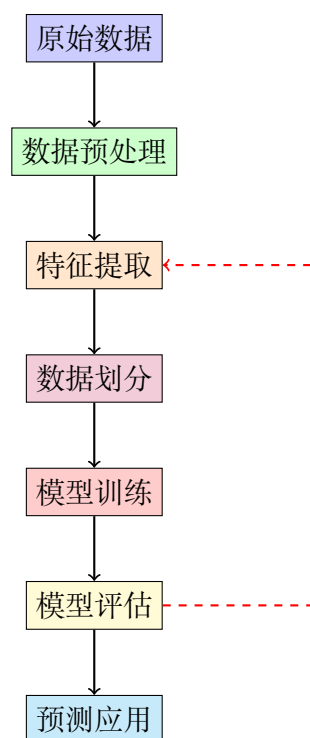


图 4: 文本分类标准流程

### 3.2.2 各步骤详解

#### 1. 数据预处理

- 文本清洗：去除 HTML 标签、特殊符号
- 分词处理：中文分词、英文词性还原
- 停用词过滤：去除无意义的高频词
- 数据平衡：处理类别不平衡问题

#### 2. 特征提取

- 选择特征表示方法（BoW、TF-IDF、词向量）
- 特征选择和降维
- 特征标准化和归一化

#### 3. 模型训练

- 选择合适的分类算法
- 超参数调优
- 交叉验证

### 3.3 3.3 评估指标 (15 分钟)

#### 3.3.1 混淆矩阵

混淆矩阵是评估分类性能的基础工具。

表 6: 二分类混淆矩阵

| 实际标签 | 预测标签    |         | 总计      |
|------|---------|---------|---------|
|      | 正类      | 负类      |         |
| 正类   | TP      | FN      | TP + FN |
| 负类   | FP      | TN      | FP + TN |
| 总计   | TP + FP | FN + TN | N       |

其中:

- **TP (True Positive):** 真正例
- **TN (True Negative):** 真负例
- **FP (False Positive):** 假正例
- **FN (False Negative):** 假负例

#### 3.3.2 核心评估指标

##### 1. 准确率 (Accuracy)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

##### 2. 精确率 (Precision)

$$Precision = \frac{TP}{TP + FP} \quad (6)$$

##### 3. 召回率 (Recall)

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

##### 4. F1 分数

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (8)$$

#### 3.3.3 多分类评估

对于多分类问题, 可以计算宏平均和微平均:

**宏平均 (Macro Average)**

$$Macro-Precision = \frac{1}{k} \sum_{i=1}^k Precision_i \quad (9)$$

**微平均 (Micro Average)**

$$Micro-Precision = \frac{\sum_{i=1}^k TP_i}{\sum_{i=1}^k (TP_i + FP_i)} \quad (10)$$

## 4 实战项目：中文情感分析系统 (75 分钟)

### 4.1 项目概述 (10 分钟)

#### 4.1.1 项目目标

构建一个中文电影评论情感分析系统，能够自动判断评论的情感倾向（正面/负面）。

#### 4.1.2 技术栈

- 数据处理：pandas, numpy
- 文本处理：jieba, re
- 特征提取：scikit-learn (TfidfVectorizer)
- 机器学习：scikit-learn (多种分类器)
- 可视化：matplotlib, seaborn
- 模型保存：pickle, joblib

### 4.2 数据准备与预处理 (20 分钟)

#### 4.2.1 数据集准备

```
1 import pandas as pd
2 import numpy as np
3 import jieba
4 import re
5 from sklearn.model_selection import train_test_split
6 from sklearn.feature_extraction.text import TfidfVectorizer
7 from sklearn.naive_bayes import MultinomialNB
8 from sklearn.svm import SVC
9 from sklearn.ensemble import RandomForestClassifier
10 from sklearn.metrics import classification_report, confusion_matrix
11 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
12 import matplotlib.pyplot as plt
13 import seaborn as sns
14 import warnings
15 warnings.filterwarnings('ignore')
16
17 # 设置中文显示
18 plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
19 plt.rcParams['axes.unicode_minus'] = False
20
21 class ChineseTextClassifier:
22     """ 中文文本分类器 """
23
24     def __init__(self):
```

```

25     """初始化分类器"""
26     self.vectorizer = None
27     self.models = {}
28     self.stop_words = self._load_stop_words()
29     self.is_fitted = False
30
31     def _load_stop_words(self):
32         """加载中文停用词"""
33         stop_words = {
34             '的', '了', '在', '是', '我', '有', '和', '就',
35             '不', '人', '都', '一', '一个', '上', '也', '很',
36             '到', '说', '要', '去', '你', '会', '着', '没有',
37             '看', '好', '自己', '这样', '那么', '能', '下',
38             '可以', '还', '比', '而', '来', '用', '从', '把',
39             '被', '跟', '与', '对', '为', '以', '所', '当',
40             '如果', '虽然', '但是', '然后', '因为', '所以'
41         }
42         return stop_words
43
44     def preprocess_text(self, text):
45         """文本预处理"""
46         if not isinstance(text, str):
47             return ""
48
49         # 清理文本
50         text = re.sub(r'[\u4e00-\u9fff\w\s]', '', text) # 保留中文和字母数字
51         text = re.sub(r'\s+', ' ', text) # 多个空格合并为一个
52         text = text.strip()
53
54         # 分词
55         words = jieba.cut(text)
56
57         # 过滤停用词和短词
58         filtered_words = [
59             word for word in words
60             if word not in self.stop_words
61             and len(word.strip()) > 1
62             and not word.isdigit()
63         ]
64
65         return ' '.join(filtered_words)
66
67     def create_sample_data(self):
68         """创建示例数据集"""
69         positive_reviews = [
70             "这部电影太精彩了，剧情跌宕起伏，演员表演出色",
71             "非常好看的电影，推荐大家观看，五星好评",
72             "导演拍得很用心，每个细节都很棒，值得收藏",
73             "演员演技在线，剧情引人入胜，是今年最佳电影",

```

```

74         "画面精美，音乐动听，整体制作水准很高",
75         "故事情节丰富，人物刻画深刻，观影体验极佳",
76         "这是一部经典之作，值得反复观看和思考",
77         "电影节奏把握得当，悬念设置巧妙，非常吸引人",
78         "特效制作精良，视觉效果震撼，技术水平一流",
79         "主题深刻，发人深省，是一部有内涵的好电影"
80     ]
81
82     negative_reviews = [
83         "这部电影实在太无聊了，浪费时间和金钱",
84         "剧情拖沓，节奏缓慢，看得让人昏昏欲睡",
85         "演员表演生硬，台词尴尬，毫无代入感",
86         "特效粗糙，制作粗制滥造，完全不值票价",
87         "故事情节老套，毫无新意，预测性太强",
88         "导演水平有限，整部电影缺乏亮点和深度",
89         "音效刺耳，画面模糊，技术问题太多",
90         "人物塑造失败，逻辑混乱，前后矛盾",
91         "过度商业化，缺乏艺术性，纯粹圈钱之作",
92         "结局草率，前面铺垫完全白费，严重失望"
93     ]
94
95     # 创建数据框
96     data = []
97     for review in positive_reviews:
98         data.append({'review': review, 'sentiment': 1, 'label': '正面'})
99     for review in negative_reviews:
100         data.append({'review': review, 'sentiment': 0, 'label': '负面'})
101
102     df = pd.DataFrame(data)
103     return df
104
105     def train_models(self, X_train, y_train):
106         """训练多个分类模型"""
107         models = {
108             'Naive Bayes': MultinomialNB(alpha=1.0),
109             'SVM': SVC(kernel='linear', C=1.0, random_state=42),
110             'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
111         }
112
113         print("开始训练模型...")
114         for name, model in models.items():
115             print(f"训练 {name}...")
116             model.fit(X_train, y_train)
117             self.models[name] = model
118
119         self.is_fitted = True
120         print("所有模型训练完成!")
121

```

```

122 def evaluate_models(self, X_test, y_test):
123     """评估所有模型性能"""
124     results = {}
125
126     for name, model in self.models.items():
127         y_pred = model.predict(X_test)
128
129         results[name] = {
130             'accuracy': accuracy_score(y_test, y_pred),
131             'precision': precision_score(y_test, y_pred),
132             'recall': recall_score(y_test, y_pred),
133             'f1': f1_score(y_test, y_pred),
134             'predictions': y_pred
135         }
136
137     return results
138
139 def plot_evaluation_results(self, results):
140     """可视化评估结果"""
141     metrics = ['accuracy', 'precision', 'recall', 'f1']
142     models = list(results.keys())
143
144     fig, axes = plt.subplots(2, 2, figsize=(15, 10))
145     fig.suptitle('模型性能对比', fontsize=16, fontweight='bold')
146
147     for i, metric in enumerate(metrics):
148         ax = axes[i//2, i%2]
149         values = [results[model][metric] for model in models]
150
151         bars = ax.bar(models, values, alpha=0.7,
152                       color=['skyblue', 'lightgreen', 'lightcoral'])
153         ax.set_title(f'{metric.upper()} 对比', fontsize=12)
154         ax.set_ylabel(metric.upper())
155         ax.set_ylim(0, 1)
156
157         # 添加数值标签
158         for bar, value in zip(bars, values):
159             ax.text(bar.get_x() + bar.get_width()/2, value + 0.01,
160                     f'{value:.3f}', ha='center', va='bottom')
161
162     plt.tight_layout()
163     plt.show()
164
165 def plot_confusion_matrices(self, results, y_test):
166     """绘制混淆矩阵"""
167     fig, axes = plt.subplots(1, 3, figsize=(15, 4))
168     fig.suptitle('混淆矩阵对比', fontsize=16, fontweight='bold')
169
170     for i, (name, result) in enumerate(results.items()):

```

```

171         cm = confusion_matrix(y_test, result['predictions'])
172
173         sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
174                     xticklabels=['负面', '正面'],
175                     yticklabels=['负面', '正面'],
176                     ax=axes[i])
177         axes[i].set_title(f'{{name}}')
178         axes[i].set_xlabel('预测标签')
179         axes[i].set_ylabel('真实标签')
180
181     plt.tight_layout()
182     plt.show()
183
184     def predict_sentiment(self, texts, model_name='Naive Bayes'):
185         """预测文本情感"""
186         if not self.is_fitted:
187             raise ValueError("模型尚未训练, 请先调用fit方法")
188
189         if isinstance(texts, str):
190             texts = [texts]
191
192         # 预处理文本
193         processed_texts = [self.preprocess_text(text) for text in texts]
194
195         # 特征提取
196         X = self.vectorizer.transform(processed_texts)
197
198         # 预测
199         model = self.models[model_name]
200         predictions = model.predict(X)
201         probabilities = model.predict_proba(X) if hasattr(model, 'predict_proba')
202             else None
203
204         results = []
205         for i, text in enumerate(texts):
206             result = {
207                 'text': text,
208                 'prediction': '正面' if predictions[i] == 1 else '负面',
209                 'confidence': probabilities[i].max() if probabilities is not None
210                     else None
211             }
212             results.append(result)
213
214         return results
215
216     # 演示完整的训练和评估流程
217     def demonstrate_sentiment_analysis():
218         """演示情感分析系统"""
219         print("=" * 60)

```

```

218 print("中文电影评论情感分析系统演示")
219 print("=" * 60)
220
221 # 1. 创建分类器实例
222 classifier = ChineseTextClassifier()
223
224 # 2. 创建示例数据
225 print("\n1. 创建示例数据集...")
226 df = classifier.create_sample_data()
227 print(f"数据集大小: {len(df)} 条评论")
228 print("\n数据样例:")
229 print(df.head())
230
231 # 3. 数据预处理
232 print("\n2. 数据预处理...")
233 df['processed_review'] = df['review'].apply(classifier.preprocess_text)
234 print("\n预处理前后对比:")
235 for i in range(2):
236     print(f"原文: {df.iloc[i]['review']}")
237     print(f"处理后: {df.iloc[i]['processed_review']}")
238     print()
239
240 # 4. 特征提取
241 print("3. 特征提取...")
242 vectorizer = TfidfVectorizer(
243     max_features=1000,
244     ngram_range=(1, 2),
245     min_df=1,
246     max_df=0.9
247 )
248
249 X = vectorizer.fit_transform(df['processed_review'])
250 y = df['sentiment']
251 classifier.vectorizer = vectorizer
252
253 print(f"特征矩阵形状: {X.shape}")
254 print(f"特征名称示例: {vectorizer.get_feature_names_out()[:10]}")
255
256 # 5. 数据划分
257 print("\n4. 数据划分...")
258 X_train, X_test, y_train, y_test = train_test_split(
259     X, y, test_size=0.3, random_state=42, stratify=y
260 )
261 print(f"训练集大小: {X_train.shape[0]}")
262 print(f"测试集大小: {X_test.shape[0]}")
263
264 # 6. 模型训练
265 print("\n5. 模型训练...")
266 classifier.train_models(X_train, y_train)

```



```

267
268 # 7. 模型评估
269 print("\n6. 模型评估...")
270 results = classifier.evaluate_models(X_test, y_test)
271
272 # 打印详细结果
273 for name, metrics in results.items():
274     print(f"\n{name} 性能:")
275     print(f"    准确率: {metrics['accuracy']:.3f}")
276     print(f"    精确率: {metrics['precision']:.3f}")
277     print(f"    召回率: {metrics['recall']:.3f}")
278     print(f"    F1分数: {metrics['f1']:.3f}")
279
280 # 8. 可视化结果
281 print("\n7. 可视化结果...")
282 classifier.plot_evaluation_results(results)
283 classifier.plot_confusion_matrices(results, y_test)
284
285 # 9. 预测新文本
286 print("\n8. 预测新文本...")
287 test_texts = [
288     "这部电影真的很棒，演员演技出色，剧情精彩",
289     "太无聊了，完全不推荐，浪费时间",
290     "一般般，有些地方不错，有些地方很差"
291 ]
292
293 for text in test_texts:
294     predictions = classifier.predict_sentiment(text)
295     for pred in predictions:
296         print(f"文本: {pred['text']}")
297         print(f"预测: {pred['prediction']}")
298         if pred['confidence']:
299             print(f"置信度: {pred['confidence']:.3f}")
300         print()
301
302 if __name__ == "__main__":
303     demonstrate_sentiment_analysis()

```

Listing 1: 数据集创建和加载

## 4.3 特征工程优化 (20 分钟)

### 4.3.1 高级特征提取技术

```

1 class AdvancedFeatureExtractor:
2     """高级特征提取器"""
3
4     def __init__(self):
5         self.tfidf_vectorizer = None

```

```

6         self.word_vectorizer = None
7         self.feature_names = []
8
9     def extract_statistical_features(self, texts):
10         """提取统计特征"""
11         features = []
12
13         for text in texts:
14             # 文本长度特征
15             char_count = len(text)
16             word_count = len(text.split())
17             avg_word_length = np.mean([len(word) for word in text.split()]) if text.
                split() else 0
18
19             # 标点符号特征
20             punctuation_count = sum(1 for char in text if char in '! ? 。 , ; : " ' '
                ( ) 【 】 《 》 ')
21             punctuation_ratio = punctuation_count / len(text) if len(text) > 0 else
                0
22
23             # 特殊词汇特征
24             positive_words = ['好', '棒', '精彩', '优秀', '推荐', '喜欢', '满意']
25             negative_words = ['差', '烂', '无聊', '失望', '糟糕', '讨厌', '后悔']
26
27             positive_count = sum(1 for word in positive_words if word in text)
28             negative_count = sum(1 for word in negative_words if word in text)
29
30             feature_vector = [
31                 char_count, word_count, avg_word_length,
32                 punctuation_count, punctuation_ratio,
33                 positive_count, negative_count
34             ]
35             features.append(feature_vector)
36
37         return np.array(features)
38
39     def extract_ngram_features(self, texts, n_range=(1, 3)):
40         """提取N-gram特征"""
41         vectorizer = TfidfVectorizer(
42             ngram_range=n_range,
43             max_features=2000,
44             min_df=2,
45             max_df=0.8,
46             sublinear_tf=True
47         )
48
49         features = vectorizer.fit_transform(texts)
50         self.tfidf_vectorizer = vectorizer
51         return features

```

```

52
53 def combine_features(self, texts):
54     """组合多种特征"""
55     # 统计特征
56     stat_features = self.extract_statistical_features(texts)
57
58     # N-gram特征
59     ngram_features = self.extract_ngram_features(texts)
60
61     # 合并特征
62     from scipy.sparse import hstack
63     combined_features = hstack([
64         ngram_features,
65         stat_features
66     ])
67
68     return combined_features
69
70 def analyze_feature_importance(self, X, y, feature_names=None):
71     """分析特征重要性"""
72     from sklearn.feature_selection import chi2
73     from sklearn.feature_selection import SelectKBest
74
75     # 卡方检验选择特征
76     selector = SelectKBest(chi2, k=20)
77     X_selected = selector.fit_transform(X, y)
78
79     # 获取特征分数
80     scores = selector.scores_
81     if feature_names is not None:
82         feature_scores = list(zip(feature_names, scores))
83         feature_scores.sort(key=lambda x: x[1], reverse=True)
84
85         print("Top 20 重要特征:")
86         for name, score in feature_scores[:20]:
87             print(f"{name}: {score:.2f}")
88
89     return X_selected, selector
90
91 # 演示高级特征提取
92 def demonstrate_advanced_features():
93     """演示高级特征提取"""
94     extractor = AdvancedFeatureExtractor()
95
96     # 示例文本
97     texts = [
98         "这部电影真的很棒!!! 演员演技出色, 剧情精彩动人。",
99         "太无聊了。。。完全不推荐, 纯粹浪费时间和金钱。",
100         "一般般吧, 有些地方不错, 但整体感觉还是差了点。"

```

```

101 ]
102
103 labels = [1, 0, 0] # 正面、负面、负面
104
105 # 提取组合特征
106 combined_features = extractor.combine_features(texts)
107 print(f"组合特征形状: {combined_features.shape}")
108
109 # 分析特征重要性
110 if extractor.tfidf_vectorizer:
111     feature_names = extractor.tfidf_vectorizer.get_feature_names_out()
112     X_selected, selector = extractor.analyze_feature_importance(
113         combined_features, labels, feature_names
114     )

```

Listing 2: 高级特征提取方法

## 4.4 模型对比与选择 (15 分钟)

### 4.4.1 多模型对比实验

```

1 class ModelComparison:
2     """模型对比工具"""
3
4     def __init__(self):
5         self.models = {
6             'Multinomial NB': MultinomialNB(),
7             'Gaussian NB': GaussianNB(),
8             'Linear SVM': SVC(kernel='linear', probability=True),
9             'RBF SVM': SVC(kernel='rbf', probability=True),
10            'Random Forest': RandomForestClassifier(n_estimators=100),
11            'Logistic Regression': LogisticRegression(max_iter=1000),
12            'Gradient Boosting': GradientBoostingClassifier()
13        }
14        self.results = {}
15
16    def cross_validate_models(self, X, y, cv=5):
17        """交叉验证评估模型"""
18        from sklearn.model_selection import cross_val_score
19        from sklearn.preprocessing import StandardScaler
20
21        # 标准化特征 (对某些算法重要)
22        if hasattr(X, 'toarray'):
23            X_dense = X.toarray()
24        else:
25            X_dense = X
26
27        scaler = StandardScaler()
28        X_scaled = scaler.fit_transform(X_dense)

```

```

29
30 for name, model in self.models.items():
31     print(f"评估 {name}...")
32
33     # 选择合适的数据格式
34     if 'NB' in name and 'Multinomial' in name:
35         X_model = X # 多项式朴素贝叶斯使用原始稀疏矩阵
36     elif 'NB' in name:
37         X_model = X_scaled # 高斯朴素贝叶斯使用标准化数据
38     else:
39         X_model = X_scaled
40
41     # 交叉验证
42     cv_scores = cross_val_score(model, X_model, y, cv=cv, scoring='f1')
43
44     self.results[name] = {
45         'cv_mean': cv_scores.mean(),
46         'cv_std': cv_scores.std(),
47         'cv_scores': cv_scores
48     }
49
50     return self.results
51
52 def plot_model_comparison(self):
53     """可视化模型对比结果"""
54     models = list(self.results.keys())
55     means = [self.results[model]['cv_mean'] for model in models]
56     stds = [self.results[model]['cv_std'] for model in models]
57
58     fig, ax = plt.subplots(figsize=(12, 6))
59
60     # 创建条形图
61     bars = ax.bar(models, means, yerr=stds, capsize=5,
62                   alpha=0.7, color='skyblue')
63
64     ax.set_title('模型性能对比 (5折交叉验证 F1分数)', fontsize=14)
65     ax.set_ylabel('F1 分数')
66     ax.set_ylim(0, 1)
67
68     # 旋转x轴标签
69     plt.xticks(rotation=45, ha='right')
70
71     # 添加数值标签
72     for bar, mean, std in zip(bars, means, stds):
73         ax.text(bar.get_x() + bar.get_width()/2, mean + std + 0.01,
74               f'{mean:.3f}±{std:.3f}', ha='center', va='bottom', fontsize=10)
75
76     plt.tight_layout()
77     plt.show()

```

```

78
79     def get_best_model(self):
80         """获取最佳模型"""
81         best_model = max(self.results.items(), key=lambda x: x[1]['cv_mean'])
82         return best_model

```

Listing 3: 模型对比实验

## 4.5 模型解释与分析 (10 分钟)

### 4.5.1 模型可解释性分析

```

1 class ModelInterpreter:
2     """模型解释工具"""
3
4     def __init__(self, model, vectorizer):
5         self.model = model
6         self.vectorizer = vectorizer
7         self.feature_names = vectorizer.get_feature_names_out()
8
9     def get_feature_importance(self, top_n=20):
10        """获取特征重要性"""
11        if hasattr(self.model, 'feature_log_prob_'):
12            # 朴素贝叶斯
13            feature_importance = np.exp(self.model.feature_log_prob_[1]) - np.exp(
                self.model.feature_log_prob_[0])
14        elif hasattr(self.model, 'coef_'):
15            # 线性模型
16            feature_importance = self.model.coef_[0]
17        elif hasattr(self.model, 'feature_importances_'):
18            # 树模型
19            feature_importance = self.model.feature_importances_
20        else:
21            return None
22
23        # 获取重要特征
24        indices = np.argsort(np.abs(feature_importance))[-top_n:]
25        important_features = [(self.feature_names[i], feature_importance[i]) for i
                               in indices]
26        important_features.sort(key=lambda x: abs(x[1]), reverse=True)
27
28        return important_features
29
30    def explain_prediction(self, text, preprocess_func=None):
31        """解释单个预测"""
32        if preprocess_func:
33            processed_text = preprocess_func(text)
34        else:
35            processed_text = text

```

```

36
37 # 获取特征向量
38 feature_vector = self.vectorizer.transform([processed_text])
39
40 # 预测
41 prediction = self.model.predict(feature_vector)[0]
42 if hasattr(self.model, 'predict_proba'):
43     prob = self.model.predict_proba(feature_vector)[0]
44 else:
45     prob = None
46
47 # 找到激活的特征
48 activated_features = []
49 feature_vector_dense = feature_vector.toarray()[0]
50
51 for i, value in enumerate(feature_vector_dense):
52     if value > 0:
53         activated_features.append((self.feature_names[i], value))
54
55 activated_features.sort(key=lambda x: x[1], reverse=True)
56
57 return {
58     'text': text,
59     'prediction': prediction,
60     'probability': prob,
61     'activated_features': activated_features[:10]
62 }
63
64 def plot_feature_importance(self, top_n=20):
65     """可视化特征重要性"""
66     important_features = self.get_feature_importance(top_n)
67
68     if important_features is None:
69         print("该模型不支持特征重要性分析")
70         return
71
72     features, importances = zip(*important_features)
73
74     fig, ax = plt.subplots(figsize=(10, 8))
75
76     # 颜色编码：正值为红色，负值为蓝色
77     colors = ['red' if imp > 0 else 'blue' for imp in importances]
78
79     bars = ax.barh(range(len(features)), importances, color=colors, alpha=0.7)
80     ax.set_yticks(range(len(features)))
81     ax.set_yticklabels(features)
82     ax.set_xlabel('特征重要性')
83     ax.set_title(f'Top {top_n} 重要特征')
84

```

```

85     # 添加图例
86     from matplotlib.patches import Patch
87     legend_elements = [Patch(facecolor='red', alpha=0.7, label='正面倾向'),
88                        Patch(facecolor='blue', alpha=0.7, label='负面倾向')]
89     ax.legend(handles=legend_elements)
90
91     plt.tight_layout()
92     plt.show()
93
94 # 完整演示代码
95 def complete_sentiment_analysis_demo():
96     """完整的情感分析演示"""
97     print("=" * 80)
98     print("完整的中文情感分析系统演示")
99     print("=" * 80)
100
101     # 1. 创建并准备数据
102     classifier = ChineseTextClassifier()
103     df = classifier.create_sample_data()
104
105     # 扩展数据集
106     additional_positive = [
107         "导演功力深厚，每个镜头都很有意义",
108         "音乐配得恰到好处，增色不少",
109         "这个演员的表演层次丰富，很有感染力",
110         "剧本写得很好，对话自然流畅",
111         "场景设计用心，服装道具都很精致"
112     ]
113
114     additional_negative = [
115         "剪辑混乱，故事讲述不清楚",
116         "演员台词功底差，表情僵硬",
117         "特效五毛钱，完全没有说服力",
118         "剧情逻辑漏洞百出，经不起推敲",
119         "节奏拖沓，很多情节都是多余的"
120     ]
121
122     for review in additional_positive:
123         df = df.append({'review': review, 'sentiment': 1, 'label': '正面'},
124                        ignore_index=True)
125     for review in additional_negative:
126         df = df.append({'review': review, 'sentiment': 0, 'label': '负面'},
127                        ignore_index=True)
128
129     print(f"扩展后数据集大小: {len(df)} 条评论")
130
131     # 2. 高级特征提取
132     extractor = AdvancedFeatureExtractor()
133     processed_texts = [classifier.preprocess_text(text) for text in df['review']]

```



```

132 X = extractor.combine_features(processed_texts)
133 y = df['sentiment']
134
135 # 3. 数据划分
136 X_train, X_test, y_train, y_test = train_test_split(
137     X, y, test_size=0.3, random_state=42, stratify=y
138 )
139
140 # 4. 模型对比
141 print("\n进行模型对比...")
142 comparison = ModelComparison()
143 results = comparison.cross_validate_models(X_train, y_train)
144 comparison.plot_model_comparison()
145
146 best_model_name, best_result = comparison.get_best_model()
147 print(f"\n最佳模型: {best_model_name}")
148 print(f"平均F1分数: {best_result['cv_mean']:.3f} ± {best_result['cv_std']:.3f}")
149
150 # 5. 训练最佳模型
151 best_model = comparison.models[best_model_name]
152 best_model.fit(X_train.toarray() if hasattr(X_train, 'toarray') else X_train,
153               y_train)
154
155 # 6. 模型解释
156 interpreter = ModelInterpreter(best_model, extractor.tfidf_vectorizer)
157 interpreter.plot_feature_importance()
158
159 # 7. 预测示例
160 test_examples = [
161     "这部电影的视觉效果令人震撼，故事情节也很吸引人",
162     "完全是在浪费时间，剧情无聊透顶",
163     "有些地方不错，但整体来说比较平庸"
164 ]
165
166 print("\n预测示例:")
167 for text in test_examples:
168     explanation = interpreter.explain_prediction(text, classifier.
169         preprocess_text)
170     print(f"\n文本: {explanation['text']}")
171     print(f"预测: {'正面' if explanation['prediction'] == 1 else '负面'}")
172     if explanation['probability'] is not None:
173         print(f"置信度: {explanation['probability'].max():.3f}")
174     print(f"激活特征: ", [f"{feat}({val:.3f})" for feat, val in explanation['
175         activated_features'][:5]])
176
177 if __name__ == "__main__":
178     complete_sentiment_analysis_demo()

```

Listing 4: 模型解释工具

## 5 总结与扩展

### 5.1 5.1 课程总结

本节课我们学习了文本特征提取和分类的核心概念：

**特征提取方法：**

- 词袋模型 (BoW)：简单但有效的基础方法
- TF-IDF：考虑词汇重要性的改进方法
- N-gram：捕获局部词序信息
- 词向量：密集表示，语义相似性

**文本分类流程：**

- 数据预处理：清洗、分词、过滤
- 特征工程：提取、选择、组合
- 模型训练：选择算法、调优参数
- 评估优化：指标计算、模型对比

**评估指标：**

- 准确率、精确率、召回率、F1 分数
- 混淆矩阵分析
- 交叉验证

### 5.2 5.2 进阶方向

**深度学习方法：**

- 循环神经网络 (RNN/LSTM/GRU)
- 卷积神经网络 (TextCNN)
- 注意力机制 (Attention)
- 预训练模型 (BERT/GPT)

**高级技术：**

- 多标签分类：一个文档多个类别
- 层次分类：类别间存在层次关系
- 少样本学习：有限标注数据的分类

- 增量学习：动态更新模型

#### **实际应用优化：**

- 类别不平衡处理
- 噪声数据清洗
- 模型压缩与加速
- 在线学习与部署

### **5.3 5.3 实践建议**

#### **特征工程最佳实践：**

1. **从简单开始：**先尝试 BoW 和 TF-IDF，再考虑复杂方法
2. **领域适应：**根据具体任务调整预处理和特征提取
3. **特征组合：**统计特征 + 文本特征 + 外部特征
4. **降维处理：**使用 PCA、LDA 等方法减少特征维度

#### **模型选择指导：**

- **数据规模小：**朴素贝叶斯、线性 SVM
- **数据规模中等：**随机森林、梯度提升
- **数据规模大：**深度学习、预训练模型
- **实时性要求高：**简单线性模型

#### **评估与调优：**

- 使用多种评估指标综合判断
- 关注类别不平衡对评估的影响
- 进行错误分析，理解模型局限性
- 定期重新训练和验证模型

## **6 课后练习**

### **6.1 6.1 基础练习**

#### **练习 1：特征提取对比**

1. 使用提供的中文文本数据集

2. 分别实现 BoW、TF-IDF、Bigram 特征提取
3. 比较不同特征在同一分类器上的性能
4. 分析各种方法的优缺点

### **练习 2：模型调优**

1. 选择一个基础分类器（如朴素贝叶斯）
2. 通过网格搜索优化超参数
3. 使用交叉验证评估模型稳定性
4. 分析参数对模型性能的影响

## **6.2 进阶练习**

### **练习 3：多分类任务**

1. 构建新闻分类数据集（体育、科技、娱乐、政治）
2. 实现完整的分类流程
3. 处理类别不平衡问题
4. 分析各类别的混淆情况

### **练习 4：特征工程创新**

1. 设计领域特定的特征（如情感词典特征）
2. 尝试特征选择算法（卡方检验、互信息）
3. 实验特征组合的效果
4. 评估计算效率和存储需求

## **6.3 项目练习**

### **项目：端到端文本分类系统**

1. **数据收集**：爬取或使用公开数据集
2. **数据标注**：设计标注规范，进行人工标注
3. **系统开发**：构建完整的分类系统
4. **性能优化**：提升模型准确率和效率
5. **部署上线**：将模型部署为 Web 服务

### **评估标准：**

- 代码质量和文档完整性 (20%)
- 特征工程的创新性 (30%)
- 模型性能指标 (30%)
- 系统可用性和扩展性 (20%)

## **7 参考资料**

### **7.1 7.1 推荐阅读**

#### **经典教材：**

- 《统计自然语言处理》- 宗成庆
- 《自然语言处理综论》- Daniel Jurafsky
- 《机器学习》- 周志华
- 《Python 自然语言处理》- Steven Bird

#### **论文推荐：**

- "Efficient Estimation of Word Representations in Vector Space" (Word2Vec)
- "Distributed Representations of Sentences and Documents" (Doc2Vec)
- "Convolutional Neural Networks for Sentence Classification" (TextCNN)
- "Attention Is All You Need" (Transformer)

### **7.2 7.2 在线资源**

#### **数据集：**

- 清华新闻分类数据集 (THUCNews)
- 复旦大学中文文本分类数据集
- 中文情感分析数据集 (ChnSentiCorp)
- Kaggle 竞赛数据集

#### **工具库：**

- scikit-learn: 机器学习基础库
- jieba: 中文分词工具
- gensim: 词向量和主题模型
- transformers: 预训练模型库

## 7.3 7.3 实验环境

推荐配置：

- Python 3.8+
- Jupyter Notebook/Lab
- 内存：8GB 以上
- 存储：SSD 硬盘

云平台：

- Google Colab：免费 GPU 资源
- Kaggle Kernels：竞赛环境
- 阿里云 PAI：企业级 ML 平台
- 腾讯云 ML：中文优化平台

## 8 附录

### 8.1 A.1 常用中文停用词表

表 7: 中文停用词示例

|   |    |    |    |   |
|---|----|----|----|---|
| 的 | 了  | 在  | 是  | 我 |
| 有 | 和  | 就  | 不  | 人 |
| 都 | 一  | 一个 | 上  | 也 |
| 很 | 到  | 说  | 要  | 去 |
| 你 | 会  | 着  | 没有 | 看 |
| 好 | 自己 | 这样 | 那么 | 能 |

### 8.2 A.2 评估指标计算公式汇总

表 8: 分类评估指标

| 指标    | 公式                                                            |
|-------|---------------------------------------------------------------|
| 准确率   | $\frac{TP+TN}{TP+TN+FP+FN}$                                   |
| 精确率   | $\frac{TP}{TP+FP}$                                            |
| 召回率   | $\frac{TP}{TP+FN}$                                            |
| F1 分数 | $\frac{2 \times Precision \times Recall}{Precision + Recall}$ |
| 特异性   | $\frac{TN}{TN+FP}$                                            |
| AUC   | $\int_0^1 TPR d(FPR)$                                         |

### 8.3 A.3 常见问题与解决方案

#### Q1: 如何处理中文分词错误？

- 使用自定义词典补充领域词汇
- 尝试不同的分词工具 (jieba、LTP、pkuseg)
- 考虑字符级别的特征提取

#### Q2: 类别不平衡如何处理？

- 数据层面：过采样、欠采样、SMOTE
- 算法层面：调整类别权重、集成方法
- 评估层面：使用 F1 分数、AUC 等指标

#### Q3: 如何提高模型泛化能力？

- 增加训练数据的多样性
- 使用正则化技术防止过拟合
- 交叉验证选择最优模型
- 特征工程减少噪声

#### Q4: 计算资源受限如何优化？

- 特征选择减少维度
- 使用简单高效的算法
- 批量处理和并行计算
- 模型压缩和量化

## 课程结语

通过本节课的学习，我们深入了解了文本特征提取和分类的核心技术。从经典的统计方法到现代的机器学习算法，从理论原理到实际应用，我们构建了完整的知识体系。

文本分类作为 NLP 的基础任务，不仅具有重要的理论价值，更有广泛的实际应用。掌握这些技术，为我们进一步学习更高级的 NLP 技术打下了坚实的基础。

在实际项目中，成功的关键往往在于：

- 深入理解业务需求和数据特点
- 选择合适的技术方案和评估指标
- 进行充分的实验和调优

- 关注模型的可解释性和可维护性

希望同学们能够将所学知识应用到实际项目中，在实践中不断提升自己的技能水平。下一节课我们将学习更高级的 NLP 技术，包括序列标注、句法分析等内容。

— 课程结束 —